Agentic AI for Business Automation (AABA)

PGDM-BDA, Term 04, 2026-27

Report on

SchemeSathi

A Single-Agent, Rules-Grounded AI Assistant that Helps Citizens Find and Claim the Government Schemes They Are Entitled To

Submitted in partial fulfilment of the requirements for the Post Graduate Diploma in Management (PGDM)

FOUNDATION FOR ORGANISATIONAL RESEARCH AND EDUCATION
FORE SCHOOL OF MANAGEMENT
NEW DELHI

Submitted to

Faculty Name: Prof. Shiva Botchu

Submitted by

Mahaveer Soni
Roll No. 065089

FORE School of Management, New Delhi

Individual Assignment · October 2026

Code: https://github.com/Mahaveersonii/schemesathi

Contents

1. Executive Summary
2. The Problem
   2.1 Benefits exist, but people do not reach them
   2.2 Why a plain chatbot is not the answer
   2.3 Who benefits
3. Objectives
4. Why a Single Agent
5. System Architecture
6. Component Design
   6.1 Data pipeline: the official source
   6.2 Rule engine
   6.3 Scheme library (RAG)
   6.4 The agent and its tools
   6.5 The guardrail
   6.6 Model choice and fallback
   6.7 The app
   6.8 Audit log
7. The Agent in Action
8. Evaluation
   8.1 Method
   8.2 Eligibility: rules versus an LLM alone
   8.3 Retrieval
   8.4 Agent end to end
9. Problems Found and Fixed
10. Business Value and Automation Impact
11. Limitations and Ethics
12. Future Work
13. Conclusion
Appendix A. How to Run
   Repository structure
References

Right-click the table and choose Update Field if page numbers are not shown.

1. Executive Summary

India runs hundreds of welfare schemes for farmers, workers, widows, students, pregnant women and people with disabilities. The benefits are real: cash every year, pensions, cheap insurance, collateral-free loans and free health care. But the people who most need them are the least able to find out which ones they qualify for, what the rules mean, which documents to carry and where to apply. A general-purpose chatbot looks like the obvious fix, but it guesses: in this project's tests an LLM working alone got eligibility wrong roughly one time in three.

SchemeSathi ("scheme companion") is a single AI agent built so that it cannot guess. A person fills in a short profile once. A **rule engine** checks them against the official eligibility rules of **32 central schemes** (95 checks, each with its official quote kept beside it) and returns *eligible*, *likely* or *not eligible* with the reason. For questions, **one CrewAI agent** plans its own calls to **five tools**: eligibility, benefit estimate, search over the official text, exact section lookup and a merged document checklist. It answers in English or Hindi. Facts come from a **retrieval (RAG) library** of 522 passages from myScheme, the Government of India portal. A **guardrail written in code** rejects any answer that:

- cites a source no tool returned;
- contradicts a rule-engine verdict;
- names a scheme or website that is not in the official text;
- states a cash total different from the computed one.

On 12 hand-labelled people × 32 schemes, the rule engine matched the labels with **F1 = 100%.** The same LLM answering alone scored **F1 = 66%**: it made 27 wrong suggestions and missed 22 schemes people qualified for. Retrieval finds the exact official section within the top three results for **83%** of 24 test questions. In end-to-end runs, **100%** of agent answers passed every guardrail check. The whole system runs on free tiers: Groq for the LLM, and Ollama and ChromaDB on a laptop.

The code, data, tests and evaluation are public at https://github.com/Mahaveersonii/schemesathi.

2. The Problem

2.1 Benefits exist, but people do not reach them

Government schemes are published, but they are written for administrators rather than citizens. Each scheme has its own eligibility rules (age bands, land in one's own name, income ceilings, BPL status, caste category, not being an income-tax payer, not being in EPFO), its own exclusions, its own document list and its own application route. A small farmer who wants to know what he can claim has to read dozens of guideline pages, many of them only in English. In practice people depend on word of mouth or on middlemen, and both miss schemes and sometimes charge for free services.

The Government's own myScheme portal brings scheme information into one place, which is a major step. It still leaves the person to read every scheme's rules and work out their own answer. The rules also interact: someone may be eligible for PM-KISAN, "likely" for crop insurance and ineligible for a pension scheme only because they are two years too old. SchemeSathi adds the missing layer: *apply the rules to me, explain the result, and tell me what to do next.*

2.2 Why a plain chatbot is not the answer

An LLM can talk fluently about schemes, but it has three failure modes that matter more here than in most domains:

- **It guesses eligibility.** In this project's baseline (section 8.2) the model offered the old-age pension to a 34-year-old, the widow pension to a married street vendor, and PM-KISAN to a tenant farmer whose land is not in his name. It also forgot MGNREGA for almost every rural person.
- **It invents details.** During development the agent wrote web addresses that do not exist (for example a made-up "pradhanmantrijeevanjyotibima.gov.in"), brought in schemes outside its library, and added up a yearly cash total that was more than eight times the real figure.
- **It cannot show its source.** A citizen, a Common Service Centre operator or an NGO worker needs to be able to show the official line to a bank or a block office.

A wrong "yes" wastes a poor family's day and bus fare at an office. A wrong "no" quietly costs them money they are owed. So the design goal was not "a better chatbot" but **an agent whose every claim can be checked**.

2.3 Who benefits

User	What SchemeSathi gives them
Citizens and families	One profile, every scheme checked, the money added up, one document list, answers in Hindi
Common Service Centre operators / village-level entrepreneurs	A screening step that takes seconds per visitor instead of reading guidelines, with official quotes to show
NGOs and self-help-group federations	Bulk screening of members; reasons for each "no" to guide follow-up (for example getting land records corrected)
Bank business correspondents and microfinance staff	Quick checks for PMJJBY, PMSBY, APY, MUDRA and SVANidhi at the point of contact

User	What SchemeSathi gives them
CSR teams and government departments	An audit log of what was asked and answered, and evidence of which rules most often block people

3. Objectives

1. Decide eligibility for 32 central schemes **deterministically**, with the official rule behind every "yes" and "no".
2. Answer free-text questions in **English or Hindi** with an agent that chooses its own tools and **cites the official text** for every fact.
3. Make it **impossible for the LLM to overrule** an eligibility verdict or to cite something it did not retrieve.
4. Turn the result into action: **money per year**, a **merged document checklist** and **where to apply**.
5. **Measure** it against a hand-labelled test set and against the same LLM working alone.
6. Keep it **free and private**: free LLM tier, local embeddings, and no personal data stored beyond the question log.

4. Why a Single Agent

The assignment asked for a single-agent design, and this problem suits one. There is one user, one conversation and one decision to make: which tools to call to answer this question for this person. Splitting that across several agents (a "researcher", an "eligibility officer", a "writer") would add hand-offs where facts can be lost or rewritten, multiply token use on a free tier, and make it harder to say which component is responsible for a wrong answer.

SchemeSathi puts the intelligence in a **single reasoning loop** (CrewAI's ReAct-style agent: think, call a tool, read the result, decide again) and puts the **reliability in deterministic tools and a code guardrail**. That split is the main design idea of the project:

Job	Done by	Why
Deciding who qualifies	Rule engine (Python)	Rules are exact; an LLM is not. Verdicts must be repeatable and explainable.
Finding facts	Retrieval over official text	Facts must come from the source, with a citation.
Choosing what to look up, explaining, translating	The LLM agent	This is where language understanding adds value.
Checking the answer	Guardrail (Python)	Self-checking by the same LLM is weak; code checks are not.

5. System Architecture

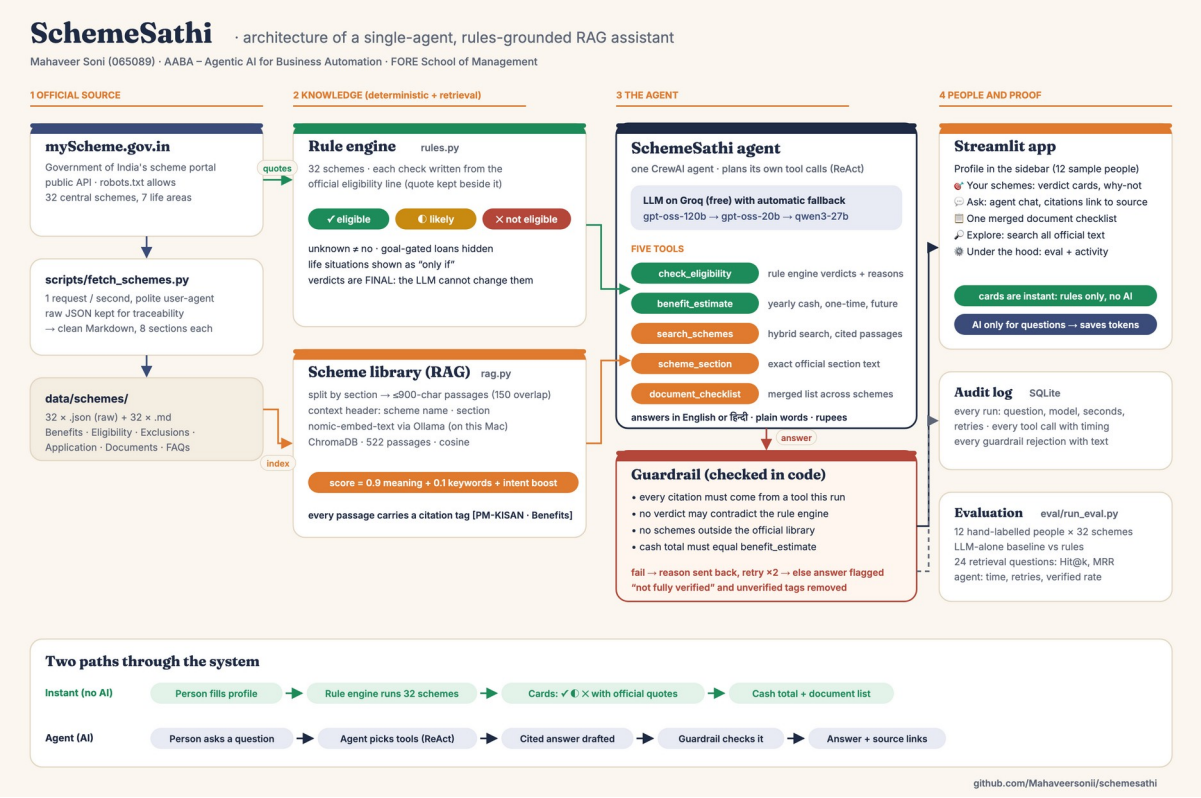


Figure 1. SchemeSathi architecture: official source → knowledge (rules + retrieval) → one agent with five tools and a guardrail → app, audit log and evaluation.

There are **two paths** through the system. The **instant path** uses no AI. As the person edits their profile, the rule engine re-evaluates all 32 schemes in milliseconds and the app shows verdict cards, the yearly cash and the document list. The **agent path** is used only when the person asks a question. The agent receives the profile and the question, calls tools, drafts an answer with citations, and the guardrail approves or rejects it. Because most value comes from the instant path, the LLM is called far less often, which matters on a free tier with a daily token limit.

6. Component Design

6.1 Data pipeline: the official source

myScheme renders scheme pages in the browser, but its public API returns the content as JSON, and its robots.txt allows crawling. `scripts/fetch_schemes.py` fetches the 32 schemes listed in `data/scheme_list.yaml` at one request per second with an identifying user-agent. It keeps the **raw JSON** for traceability and writes a **clean Markdown file** per scheme with fixed sections: Details, Benefits, Eligibility, Exclusions, Application Process, Documents Required, FAQs and References. Some schemes store their document list only as a rich-text tree rather than Markdown. The first version silently lost the documents for 12 schemes, so a converter for that format was added, and the Markdown can be rebuilt offline from the saved JSON (`--rebuild`).

Life area	Schemes
Farmers	PM-KISAN, PM Kisan Maandhan, Kisan Credit Card, PM Fasal Bima
Health and insurance	Health Minister's Discretionary Grant, Nikshay Poshan (TB), PM Jeevan Jyoti Bima, PM Suraksha Bima
Housing and household	PMAY-Urban, PMAY-Gramin, PM Ujjwala (LPG), PM Jan Dhan
Entrepreneurs and workers	MUDRA, PM SVANidhi, Stand-Up India, PMEGP, PM Vishwakarma, PM Shram Yogi Maan-dhan, MGNREGA
Pensions	Atal Pension Yojana, Old Age (IGNOAPS), Widow (IGNWPS) and Disability (IGNDPS) pensions
Women and children	PM Matru Vandana, Janani Shishu Suraksha Karyakram, Janani Suraksha Yojana
Students and youth	Central Sector Scholarship, PM YASASVI, Post-Matric (Disability), NAPS apprenticeships, PMKVY, Skill Loan

Table 1. The 32 central schemes covered, by life area.

6.2 Rule engine

`schemesathi/rules.py` describes each scheme as a list of **checks**. Each check is a plain-language label, a test on the profile, and the official sentence it comes from. A check answers *True*, *False* or *unknown* (the person has not said, or only an office can verify it). The verdict logic is deliberately simple and explainable:

Verdict	When	Shown to the person as
Eligible	Every check passed	Green card, all reasons ticked
Likely	Nothing failed, but something is unknown or must be confirmed at the office (for example SECC listing, not being a practising professional)	Amber card with "Confirm at the office"
Not eligible	At least one check failed	Red card with the failed rule and its official quote

Three refinements came out of testing with realistic people:

- **Goal-gated schemes.** A business loan is not "failed" just because the person does not want a loan; it is shown separately as not their goal.

- **Life situations.** A maternity scheme is not "likely" for someone who simply did not mention a pregnancy. Schemes that depend on pregnancy, widowhood, disability, TB treatment or studying appear under "only if this applies to you" and are not counted in the totals.
- **Official-text corrections.** PM Kisan Maandhan needs the farmer's name in the land records. PM-SYM covers unorganised workers, not homemakers.

Money is stated **only where the official text gives an amount** (for example ₹6,000 a year for PM-KISAN). The widow pension's amount is set by each state and is not on myScheme, so SchemeSathi says exactly that rather than invent a figure.

6.3 Scheme library (RAG)

`schemesathi/rag.py` splits each scheme file by section, then into passages of at most 900 characters with 150 characters of overlap. Long paragraphs are cut at sentence ends. Each passage is embedded with a short header ("scheme name (short name) · section") so that a passage like "the loan is repaid in 12 months" still carries which scheme and section it belongs to. Embeddings use **nomic-embed-text through Ollama on the laptop** (free, and the text never leaves the machine), stored in **ChromaDB** with cosine distance: 522 passages in total.

Search blends three signals: **0.9 × meaning similarity + 0.1 × keyword overlap + a 0.05 boost** when the passage's section matches the question's intent. Intent comes from simple patterns: "documents" → Documents Required, "how do I apply" → Application Process, "who can" → Eligibility, "how much" → Benefits. Every passage is returned with a citation tag such as **[PM-KISAN · Benefits]** and a link to the scheme's myScheme page. The same module merges document lists across schemes, so "Aadhaar Card", "Copy of Aadhaar" and "Aadhar card" become one item with a count of how many schemes need it.

6.4 The agent and its tools

`schemesathi/agent.py` builds one CrewAI agent with the role of a government-scheme guide. Its instructions set the order of work: check eligibility first; use the search tools for facts; cite every fact with a tag a tool returned; quote the computed cash total; and never invent offices, websites or documents. The agent decides how many tool calls to make (at most 8 iterations). Its five tools (`schemesathi/tools.py`) each return short JSON, are timed and logged, and never crash the run:

Tool	What it returns	Backed by
check_eligibility	Schemes that fit (with what to confirm), schemes not eligible (with the failed rule), "only if" and off-goal schemes, plus citation tags	Rule engine
benefit_estimate	Yearly cash total and each fitting scheme's value in words	Rule engine
search_schemes	Top 4 official passages for a question, optionally within one scheme	RAG library
scheme_section	Exact text of one section (Benefits, Eligibility, Exclusions, Application Process, Documents Required)	Scheme files
document_checklist	One merged document list across several schemes, with citation tags	Scheme files

6.5 The guardrail

CrewAI lets a task carry a guardrail function that sees the agent's final answer. If the function rejects the answer, its reason is sent back and the agent tries again (up to two retries). SchemeSathi's guardrail is ordinary Python, so it is fast, free and cannot be talked out of its rules:

Check	Catches
Every citation tag was returned by a tool in this run	Made-up sources
No sentence calls a scheme eligible when the rules say not eligible, or the reverse	The LLM overruling the rule engine
Every scheme-like acronym is in the library or in the text the tools returned	Schemes outside the library (it once described NPS with a wrong age rule)
Every web address appears in the retrieved official text, or is myscheme.gov.in	Invented websites
A question about documents or applying must have used the matching tool	Answers from memory
A stated yearly total equals benefit_estimate's figure	LLM arithmetic (it once claimed ₹53,600 instead of ₹6,000)

Before checking, the answer is **normalised**. The model often writes PM-KISAN · Benefits with Chinese brackets and non-breaking hyphens, which looks correct to a person but failed exact matching in the first version. If the retries run out, SchemeSathi does not throw the answer away. It keeps the last draft, removes any unverified citation, marks the answer *"not fully verified"* in the app, and logs the event.

6.6 Model choice and fallback

The agent runs on **Groq's free API**, which serves open-weight models very fast. The chain is **gpt-oss-120b → gpt-oss-20b → Qwen3**. If a model hits its daily token limit or fails, the agent moves to the next one immediately (CrewAI's internal waiting is switched off). A one-off malformed tool call gets one retry on the same model. Groq is reached through CrewAI's native OpenAI-compatible client, because the default path sends a field Groq rejects. On gpt-oss-20b a typical answer takes **2 to 15 seconds**; answers that needed guardrail retries took up to about 90 seconds.

6.7 The app

`app/streamlit_app.py` uses a warm civic design: paper background, deep indigo text, saffron and leaf-green accents, serif headings, and Devanagari support. Twelve sample people can be loaded with one click. The five tabs are: *Your schemes* (verdict cards, a "not eligible and why" view and an "only if" view), *Ask SchemeSathi* (agent chat with clickable source chips, the tools used, model, time and guardrail retries), *Document checklist* (merged, downloadable), *Explore* (search and read the official text) and *Under the hood* (architecture, evaluation and the agent's activity from the log).

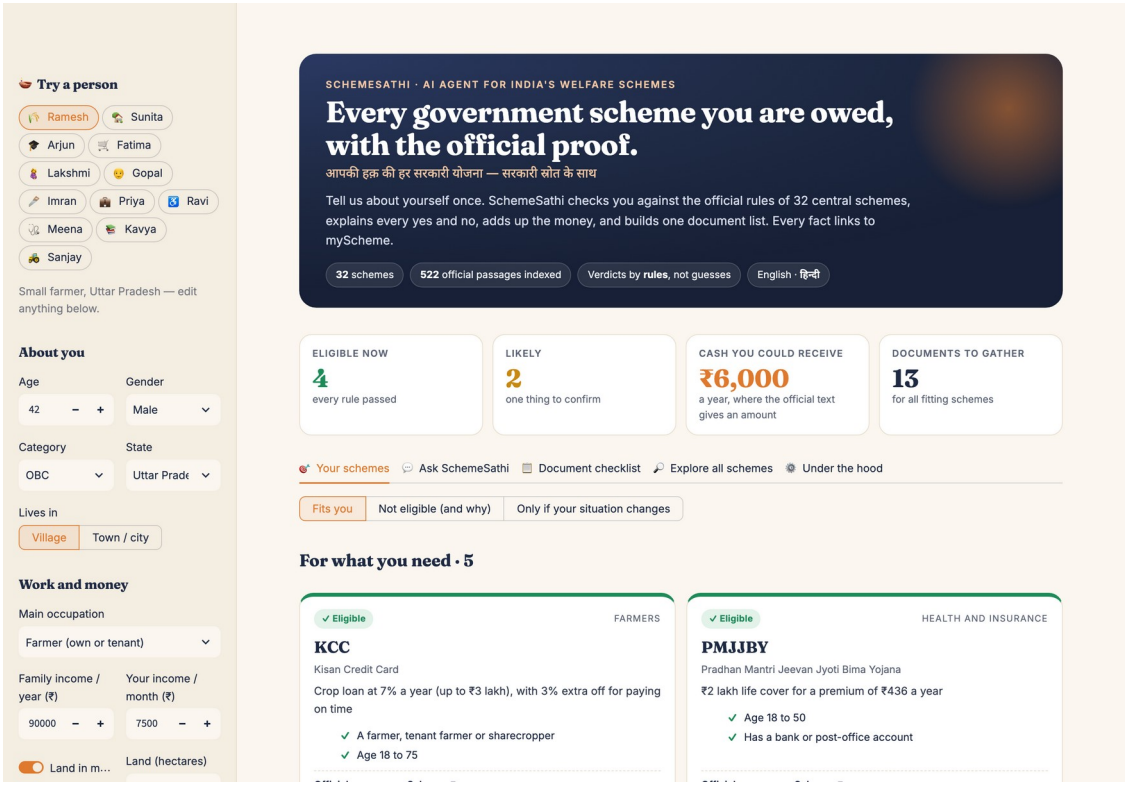


Figure 2. Home: the profile on the left, the instant verdict cards and totals on the right (sample person: Ramesh, small farmer).

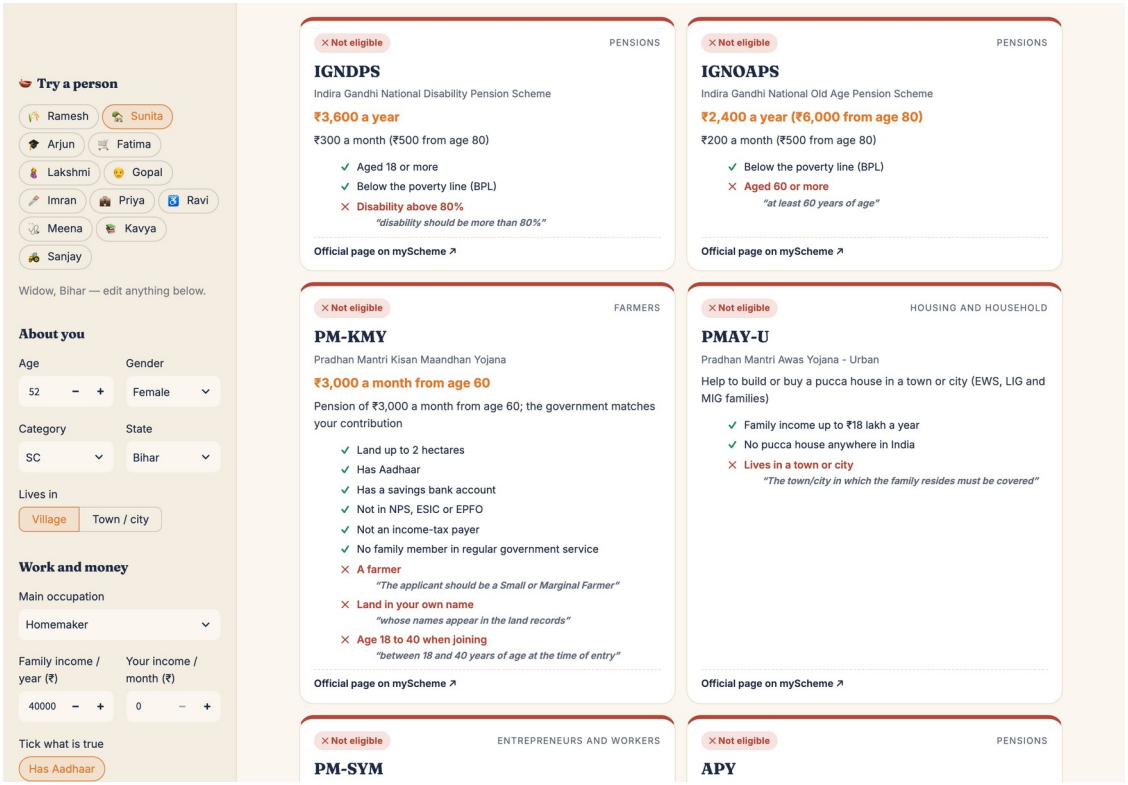


Figure 3. "Not eligible (and why)": the failed rule in red with its official quote.

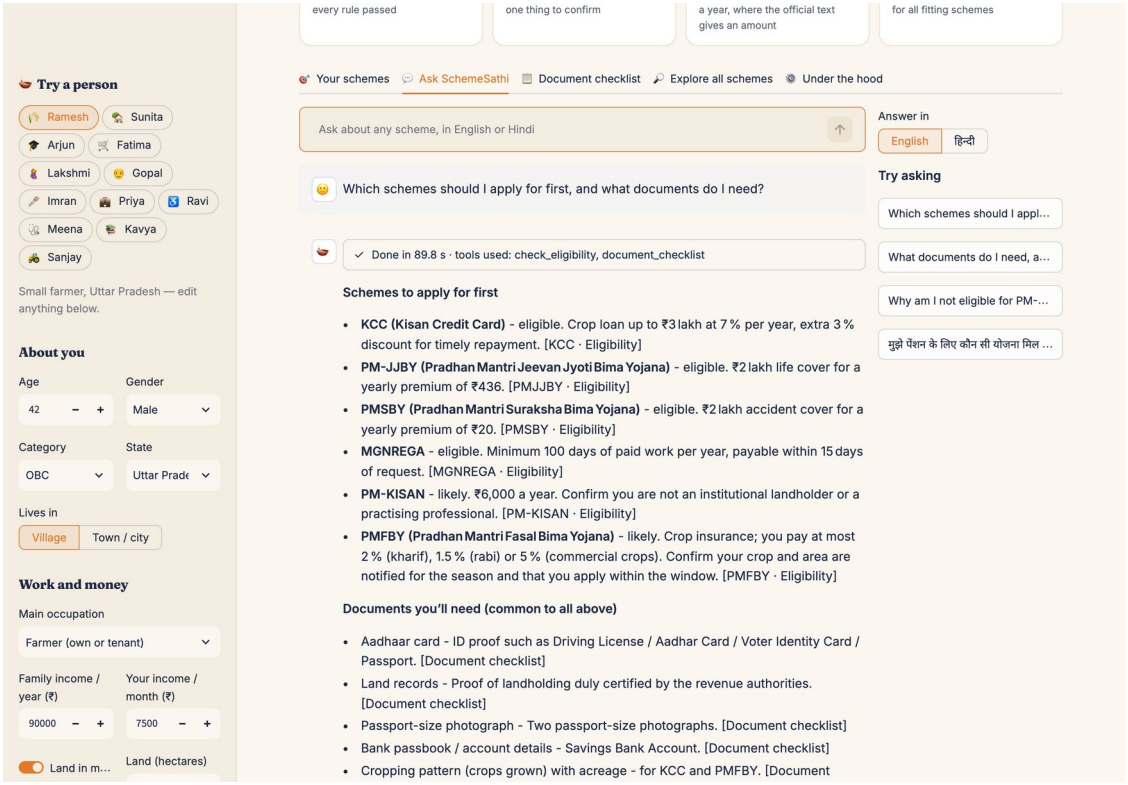


Figure 4. The agent's answer, with tools used, citations linking to mySchemе, model, time and guardrail retries.

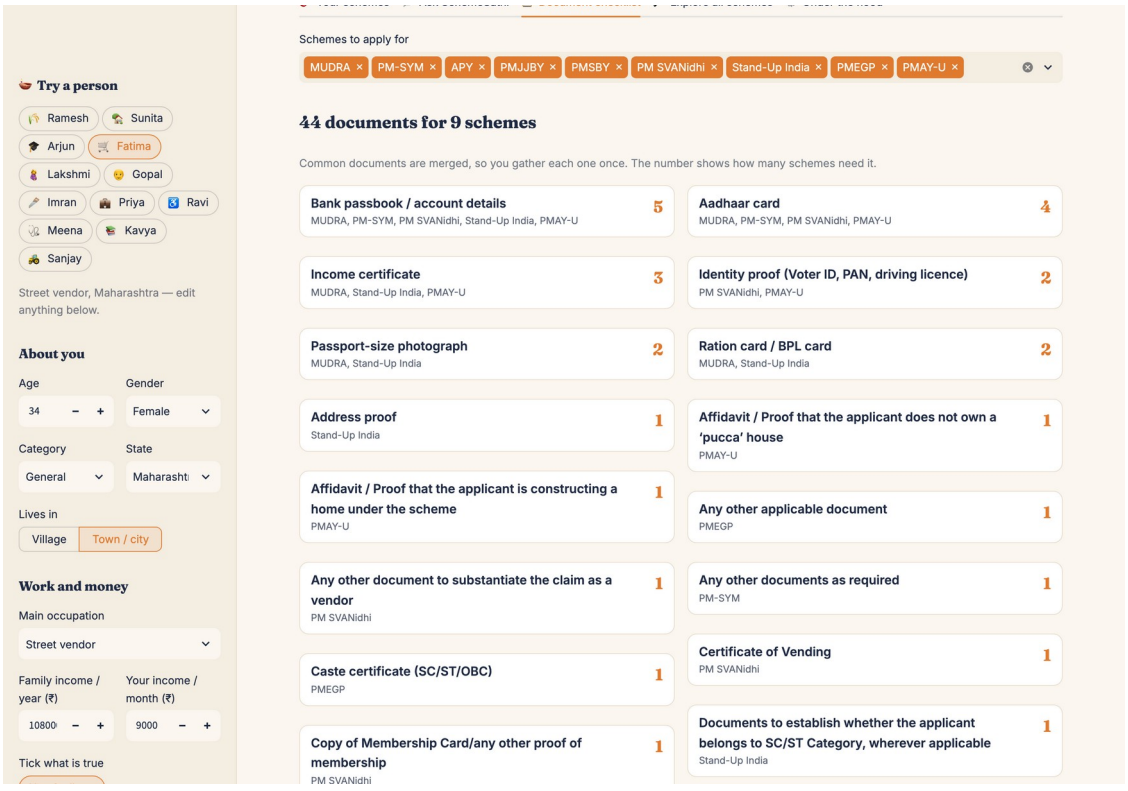


Figure 5. One merged document checklist for nine schemes (sample person: Fatima, street vendor).

6.8 Audit log

`schemesathi/log.py` writes every run (question, language, model, status, seconds, retries, answer) and every event (tool calls with arguments and timing, model fallbacks, guardrail rejections with the rejected

text) to SQLite. This is what an organisation using SchemeSathi would need for accountability, and it is how most of the problems in section 9 were found.

7. The Agent in Action

A real run from the audit log shows the loop and the guardrail at work. Profile: Ramesh, 42, small farmer in Uttar Pradesh, 1.2 ha in his own name. Question: *"Which schemes should I apply for first, and what documents do I need?"*

```
1  gpt-oss-120b          → daily token limit reached, fall back
2  gpt-oss-20b  thinks   → call check_eligibility(scope='all')
3                        ← fits: KCC, PMJJBY, PMSBY, MGNREGA (eligible); PM-KISAN, PMFBY (likely)
4  drafts answer         → GUARDRAIL: 'www.pradhanmantrijeevanjyotibima.gov.in' is not in the
                           official text you retrieved. Remove it.
5  thinks again          → call check_eligibility, document_checklist('KCC,PM-KISAN,PMFBY,...')
6  drafts answer         → GUARDRAIL: same invented address. Remove it.
7  calls tools again     → third draft: citations real, no invented links, verdicts match → ACCEPTED
   89.8 s · 2 guardrail retries · 6 official sources
```

The invented website looks plausible, and a person could easily have trusted it. That kind of error is why the guardrail is written in code rather than left to the model's judgement. The final answer lists the six fitting schemes with their verdicts, the merged documents and the next steps, each fact tagged to its official section.

8. Evaluation

8.1 Method

`eval/run_eval.py` runs four measurements and writes `eval/results.json`, which the app also displays.

- **Eligibility.** 12 realistic people (farmer, widow, student, street vendor, pregnant woman, retired person, carpenter, salaried professional, person with 85% disability, domestic worker in TB treatment, a Class 12 student without a bank account, a tenant farmer). For each, the set of schemes they should be shown was written by hand from the official eligibility text: 12 × 32 = 384 decisions.
- **LLM-alone baseline.** The same model the agent uses (gpt-oss-20b) gets the same profile and the list of 32 schemes, with no tools, and picks the schemes the person qualifies for.
- **Retrieval.** 24 questions a person might ask, each with the scheme and section where the answer is. Measured: Hit@1, Hit@3, mean reciprocal rank (MRR), and whether the top result is at least the right scheme.
- **Agent end to end.** Four people ask "Which schemes can I get, how much money a year, and what should I do first?". Measured: time, guardrail retries, verified answers, and whether all fitting schemes and no non-fitting schemes are mentioned.

8.2 Eligibility: rules versus an LLM alone

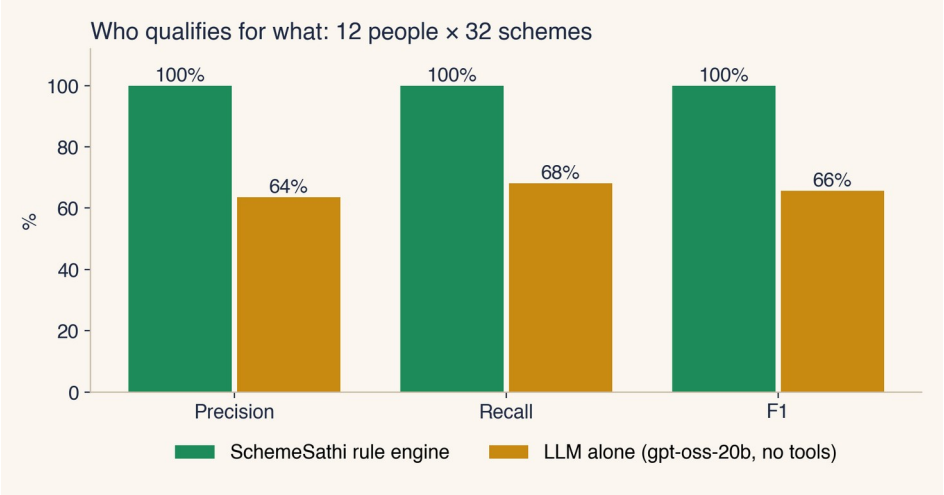


Figure 6. Precision, recall and F1 on 384 person-scheme decisions.

The rule engine matched the hand labels exactly (precision, recall and F1 all 100%). The LLM alone scored precision 64%, recall 68% and F1 66%: 47 correct, **27 wrong suggestions** and **22 missed schemes**. Its mistakes are the kind that hurt real people:

Person	LLM wrongly said yes to	LLM missed
Ramesh	APY, IGNOAPS, PM-KMY	MGNREGA
Sunita	APY, IGNOAPS	MGNREGA, PMSBY
Arjun	Skill Loan	APY, PMJJBY, PMSBY
Fatima	IGNDPS, IGNOAPS, IGNWPS, PM Vishwakarma	PMAY-U, PMJJBY, PMSBY

Person	LLM wrongly said yes to	LLM missed
Lakshmi	HMDG, Nikshay Poshan, PMAY-U, PMJDY	APY, MGNREGA
Gopal	APY, HMDG, PMJJBY	MGNREGA
Imran	IGNOAPS, PM SVANidhi, Stand-Up India	PMJJBY, PMSBY
Ravi	IGNOAPS, IGNWPS, Skill Loan	MGNREGA, PMAY-G, PMJJBY, PMSBY
Meena	—	PMAY-U
Kavya	PM-USP CSSS	—
Sanjay	IGNOAPS, PM-KISAN, PM-KMY	MGNREGA, PMJJBY, PMSBY

Table 2. Every error the LLM made working alone (gpt-oss-20b).

Patterns: the LLM handed out the **old-age pension (IGNOAPS) to people aged 34 to 42**, offered **PM-KISAN and PM-KMY to a tenant farmer** without land in his name, and **missed MGNREGA for most rural adults** and the low-cost insurance schemes PMJJBY and PMSBY for many people. The rule engine's perfect score needs one honest caveat: the labels and the rules were both written by the author from the same official text. Two rule errors (PM-KMY land records and PM-SYM homemakers) were found and fixed *while* writing the personas. The test shows that the engine applies the official rules consistently; an independent audit by a scheme officer would be the next step.

8.3 Retrieval

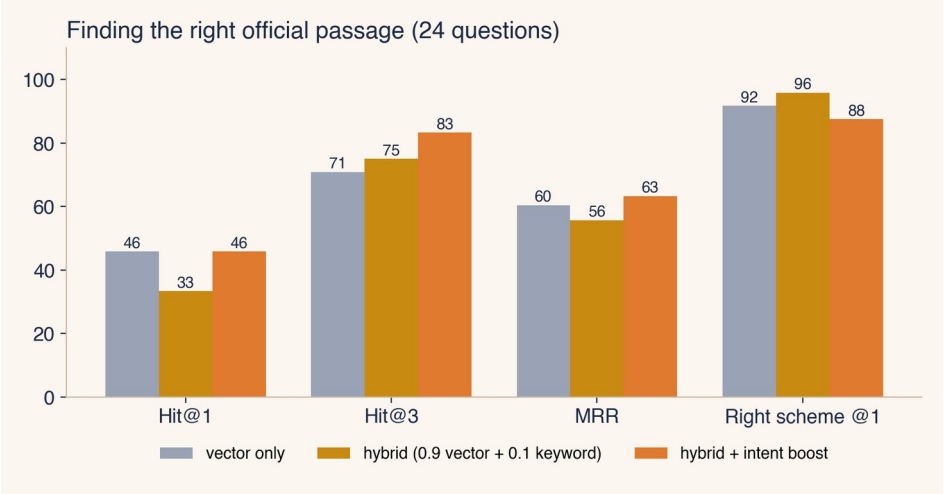


Figure 7. Retrieval variants on 24 questions.

Variant	Hit@1	Hit@3	MRR	Right scheme at #1
vector only	46%	71%	0.60	92%
hybrid (0.9 vector + 0.1 keyword)	33%	75%	0.56	96%
hybrid + intent boost	46%	83%	0.63	88%

Table 3. Exact-section retrieval and scheme-level accuracy.

Pure vector search puts the right *scheme* first for 92% of questions, but often returns the scheme's Details or FAQ passage rather than the exact section. Adding keywords helps find the scheme but on its own lowers exact-section accuracy. The **intent boost** gives the best Hit@3 (83%) and MRR (0.63), so it is the

default. In practice the agent also has scheme_section for exact lookups, which makes the agent less dependent on the top-1 result. The weights were tuned on these same 24 questions, so the numbers are optimistic; a separate held-out set would be needed for a fair estimate.

8.4 Agent end to end

Person	Model	Seconds	Guardrail retries	Verified	Citations	Fitting schemes mentioned	Wrong schemes mentioned
Ramesh	gpt-oss-20b	2	0	Yes	6	6/6	none
Sunita	gpt-oss-20b	52.5	0	Yes	5	5/5	none
Arjun	gpt-oss-20b	1.7	0	Yes	7	7/7	none
Fatima	gpt-oss-20b	4.9	0	Yes	9	9/9	none

Table 4. Four end-to-end agent runs (after the fixes in section 9).

All four answers passed every guardrail check, mentioned every scheme the person fits, and mentioned no scheme they do not fit. The average time was 15.3 seconds. One run took 52 seconds because the first model had reached its daily limit. Before the last guardrail fixes, one of four runs (Fatima) ended as "not fully verified" because the guardrail wrongly treated the housing categories EWS, LIG and MIG as unknown schemes. That false alarm led to the rule that any term appearing in the retrieved official text is allowed.

9. Problems Found and Fixed

Building an agent that can be trusted was mostly a matter of finding failures and closing them. The log and the evaluation exposed these:

Problem	How it showed up	Fix
Correct citations rejected	Every answer failed; the model writes PM-KISAN · Benefits	Normalise brackets and hyphens before checking
Documents missing for 12 schemes	Empty document lists for PM-KISAN, KCC, MGNREGA and others	Converter for myScheme's rich-text format; offline rebuild
Maternity and widow schemes "likely" for a 42-year-old man	Rule-engine output for the farmer persona	Life-situation logic; gender defaults
Invented cash total (₹53,600 vs ₹6,000)	Agent answer	Guardrail check plus instruction to quote benefit_estimate
Answers about schemes outside the library (NPS)	Hindi test question	Unknown-scheme guardrail
False alarm on EWS / LIG / MIG	Evaluation run marked unverified	Allow terms seen in the retrieved official text
Invented websites	Screenshot run	Web-address guardrail
Documents answered from memory	Screenshot run	Require the document tool for document questions
Very long paragraphs never split	Unit test	Sentence-aware chunking
Keyword blending hurt exact-section retrieval	Retrieval evaluation	Question-intent boost; weights re-tuned
Model daily token limit	429 errors; slow retries	Immediate fallback down the model chain

All of these are covered by the **39 offline unit tests** in `tests/test_core.py`, which need neither an LLM nor the embedding server, so they run in about a second.

10. Business Value and Automation Impact

- **From reading to deciding.** Screening a person against 32 schemes by hand means reading 32 sets of guidelines. SchemeSathi applies 95 rules in milliseconds and explains each result, so an operator can screen a visitor during a short counter conversation.
- **Fewer wasted trips.** The merged checklist (for example 44 documents for 9 schemes, with the bank passbook and Aadhaar needed by the most schemes) means people bring the right papers the first time.
- **Money made visible.** Showing "₹6,000 a year" or "₹2 lakh cover for ₹20 a year" turns an abstract scheme into a decision a family can make.
- **Trust through sources.** Every claim links to the official page, and every "no" quotes the rule. That protects people from middlemen and lets field workers defend an answer at a bank or office.
- **Near-zero running cost.** The deterministic path needs no AI at all. The agent runs on a free LLM tier, and embeddings run locally. A CSC or NGO could run it on one laptop.
- **Accountability.** The audit log records what was asked, which tools ran, which model answered and every guardrail rejection, which is what a department or CSR funder would need.
- **Reusable pattern.** "Rules decide, retrieval informs, the agent explains, code verifies" applies to any rule-heavy service: insurance claims, loan pre-screening, tax benefits and HR policy questions.

11. Limitations and Ethics

- **Coverage.** Only 32 central schemes; most states run additional schemes. The design scales by adding scheme files and checks.
- **"Likely" still needs an office.** Some conditions (SECC lists, land records, notified crops) can only be confirmed by an authority. SchemeSathi says so rather than hiding it.
- **Rule maintenance.** Schemes change. The raw JSON is dated and the fetch script can be re-run, but the checks need a human to review changes.
- **Remaining LLM risk.** The guardrail catches the errors it is designed for. A plausible but wrong *instruction* (for example naming a different office to visit) can still pass, which is why the app always shows the official links and labels itself a student project, not an official service.
- **Labels written by the author.** The evaluation shows consistency with the official text, not independent ground truth.
- **Privacy.** The profile stays in the browser session. Only the question and the profile summary go to the LLM provider, and the log stores questions and answers but no names. A deployment should add consent and retention rules.
- **Respect for the source.** The data was collected at one request per second through a public API that robots.txt allows, with an identifying user-agent, and every answer credits myScheme.

12. Future Work

- State schemes and more central schemes, with a change detector that re-fetches and flags rules to review.
- Voice input and output in Hindi and other Indian languages for people who cannot read comfortably.
- An independent evaluation with labels from scheme officers or CSC operators, and a held-out retrieval test set.
- Bulk screening from a CSV for NGOs and self-help-group federations, with an aggregate "most common blocking rule" report.
- A WhatsApp front end, the channel most people actually use.

13. Conclusion

SchemeSathi shows that a single agent can be both useful and trustworthy when it is built around what LLMs are good at and kept away from what they are bad at. The LLM plans, looks up, explains and translates. Deterministic code decides eligibility, does the arithmetic and checks every answer. Against the same model working alone, which got about a third of eligibility decisions wrong (F1 66%), the rules-grounded design matched the official criteria on every test person and produced answers in which every fact links to the Government's own text. The system is free to run, open source and ready for a Common Service Centre or NGO to try.

Appendix A. How to Run

```
git clone https://github.com/Mahaveersonii/schemesathi && cd schemesathi
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env              # paste a free Groq API key
ollama pull nomic-embed-text
python -c "from schemesathi import rag; print(rag.index())"
streamlit run app/streamlit_app.py
pytest -q                         # 39 offline tests
python eval/run_eval.py --baseline --agent 4
```

Repository structure

Path	Contents
schemesathi/rules.py	Rule engine: 32 schemes, 95 checks with official quotes
schemesathi/rag.py	Chunking, embeddings, ChromaDB, hybrid search with intent boost, document merging
schemesathi/tools.py	The five agent tools
schemesathi/agent.py	Agent, prompt, guardrail, model fallback
schemesathi/llm.py, config.py, log.py	Groq models, settings, SQLite audit log
schemesathi/personas.py	12 sample people with hand-written expected schemes
scripts/fetch_schemes.py	Fetch from myScheme; rebuild Markdown offline
app/	Streamlit app and design system
eval/	Evaluation script, retrieval questions, results
tests/test_core.py	39 offline unit tests
docs/	Architecture diagram, screenshots, report

References

1. myScheme, National Platform for Government Schemes, Government of India. https://www.myscheme.gov.in (scheme text fetched October 2026).

2. CrewAI documentation: agents, tasks, tools and task guardrails. https://docs.crewai.com

3. Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

4. Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

5. Nussbaum, Z. et al. (2024). Nomic Embed: Training a Reproducible Long Context Text Embedder. (nomic-embed-text)

6. Groq API documentation (OpenAI-compatible endpoint, free-tier rate limits). https://console.groq.com/docs

7. ChromaDB documentation. https://docs.trychroma.com

8. Project repository: https://github.com/Mahaveersonii/schemesathi